



Season 2026/2027

# Work Order – IFS09 Control Board

**Department: Electronics & Control**

**Author: Andrés Sánchez, Carlota Treviño & Inés Pacheco**

**Date: 3 September 2026**

**Revision: 1.1 (Onboarding Edition)**

CONFIDENTIALITY NOTICE: the contents of this document are intended solely for use within the team and its members, may contain confidential or privileged information and may be legally protected from disclosure.

# 1. Welcome to ISC Electronics: The Journey from Mingo to Tronik

Welcome to the ISC Electronics & Control Subsystem! If you are reading this, your journey from the introductory recruitment phase ('Mingo') to designing real competition embedded electronic hardware for our IFS09 single-seater electric race car has officially begun. Don't worry if you have never designed a PCB before! This Work Order is designed as a comprehensive onboarding roadmap to guide you through KiCad, GitHub, and automotive PCB design practices.

## 1.1 The Two Pillars: Work Order (WO) vs. Design Report (DR)

In Formula Student, no board is ever manufactured without following our two-step engineering lifecycle:

- **1. The Work Order (WO):** The engineering contract & requirement brief. It defines what the board must do, input voltages, connectors, and bench tests it must pass before you open KiCad.
- **2. The Design Report (DR):** The technical defense document written after finishing your PCB in KiCad. It explains how you solved the requirements, why you chose specific component values, and presents your schematic exports, 2D/3D renders, and BOM before we order the PCB.

## 1.2 Team Dynamic & Feedback Loop

You are not expected to get every trace and pin assignment right on day one! We work in 1-week feedback loops where we review your progress, answer questions, and help you route. The objective of this first project is for you to become confident with KiCad, understand our Git workflow, and master good design habits before we move on to key vehicle boards like the ECU, the Telemetry Node, and the Dash (driver dashboard).

# 2. KiCad Setup & Component Libraries

Before opening KiCad, ensure you have the proper tools and libraries installed:

- **KiCad 8.x Download:** Official open-source EDA suite. Download and install KiCad 8.x (or latest stable) from <https://www.kicad.org/download/>
- **Recommended Community Libraries:** Open the Plugin and Content Manager in KiCad and install: (1) 4ms Kicad Libraries (audio/analog/DSP footprints) and (2) Alternate Kicad Library (AKL) for clean, modernized schematic symbols.

## 2.1 The Official Team Component Library: IFS-KicadLib

Our team maintains its own centralized repository containing verified symbols and footprints for all components used in our race cars (automotive connectors, STM32 microcontrollers, CAN transceivers, TVS diodes, and team-standard SMD passives):

- **Repository Link:** <https://github.com/isc-fs/IFS-KicadLib>

Golden Rule: Before searching for a symbol in KiCad's default libraries or making a custom footprint from scratch, ALWAYS check if it already exists in IFS-KicadLib!

- **How to Link Symbols:** Preferences -> Manage Symbol Libraries -> Global Libraries -> Add IFS-KicadLib.kicad\_sym
- **How to Link Footprints:** Preferences -> Manage Footprint Libraries -> Global Libraries -> Add .pretty footprint folders

## 3. How to Work with GitHub (HW & SW in the Same Repository)

In automotive embedded systems, hardware and firmware are inseparable. A change in the hardware (e.g. reassigning a pin or changing a transceiver) immediately impacts the firmware running on that board. To ensure complete traceability, we store hardware design files and embedded firmware together in the same GitHub repository.

### 3.1 Mandatory Action: Create Your Own Repository for this Board

As your very first step in this onboarding project, you must create a dedicated GitHub repository of your own to host all your work on this board:

- **Repository Naming:** Name your repository following team standards: IFS09-CE-ControlBoard\_<YourName> (or initials).
- **Template Setup:** Initialize it using the team template at <https://github.com/isc-fs/plantilla> or mirror the structure of IFS09-TE-TeleReceiver.
- **Continuous Commits:** Every time you add a subcircuit, place components, or write test firmware, commit and push your changes with clear messages.
- **Share with Mentors:** Share your repository link with Carlota, Félix, Inés, or Andrés so we can track your progress and give you feedback during our weekly reviews!

### 3.2 The Reference Model: IFS09-TE-TeleReceiver

Inspect our IFS09 Telemetry Receiver repository as the gold standard for this unified repository structure: <https://github.com/isc-fs/IFS09-TE-TeleReceiver>

### 3.3 Standard Repository File Structure

- **docs/images/:** Contains exported schematic previews, 2D PCB routing layers, and 3D mechanical renders.
- **<Project\_Name>/:** Holds the KiCad project files (.kicad\_pro, .kicad\_sch, .kicad\_pcb).
- **SW/:** Embedded firmware sketches, STM32CubeIDE project files, and hardware drivers.
- **WO\_<ID>\_<BoardName>.pdf:** Requirements, engineering brief, and test criteria.
- **DR\_<ID>\_<BoardName>.pdf:** Technical defense, calculations, BOM, and post-assembly test results.
- **Schematics-<Version>.pdf:** Latest high-resolution vector PDF export of the schematic sheets.

### 3.4 Git Branching & Pull Request Rules

- **Branching:** Main holds only validated releases. All active design occurs on feature branches (feat/<name>).
- **Commit Messages:** Use clear prefixes: 'hw: add 100nF decoupling capacitors' or 'sw: update CAN baud rate'.
- **PR Checklist:** Before opening a PR to dev/main, ensure 0 KiCad ERC errors, 0 DRC errors, and export an updated Schematics-X.X.pdf.

## 4. How to Start in KiCad: The Schematic Editor

When you open your KiCad project, click on the Schematic Editor (Editor de esquemas). This is where your design starts: here we define the electrical logic before routing physical copper tracks.

### 4.1 Essential Shortcuts & Right Sidebar Tools

- **[Esc] Pointer / Selection:** Select and inspect elements.
- **[A] Add Symbol:** (Op-amp triangle icon) Open the component library browser to search for parts.
- **[W] Add Wire:** (Wire icon) Draw electrical connection lines between symbol pins.
- **[P] Add Power / GND:** (Ground icon) Place power supply rails (+24V, +5V, +3V3) and ground nodes (GND).
- **[L] Add Net Label:** (Tag icon) Name an electrical net. Any two wires that share the same net label are electrically connected without needing crossing wires!
- **[Q] No-Connect Flag:** Mark intentionally unused microcontroller pins.

### 4.2 How to Choose and Place Components

- **Resistors & Capacitors:** Search for generic 'R' or 'C'. Place them and edit the value (e.g. 10k, 100nF, 4.7uF). Small SMD passives use 0603 or 0805 packages.
- **External Elements:** You don't place pushbuttons or screens directly on the board schematic if they are external to the PCB! Instead, place the connector that will mate with the external wire harness and use net labels (e.g., BTN\_SELECT, I2C\_SCL).

## 5. The Mission: IFS09 Control Board Architecture

You will design an automotive-grade IFS09 Control Board (ECU Node). In our race car, this board acts as an intelligent microcontroller node connected to the car's 24V Low Voltage battery and two CAN communication networks to read sensors and drive vehicle actuators.

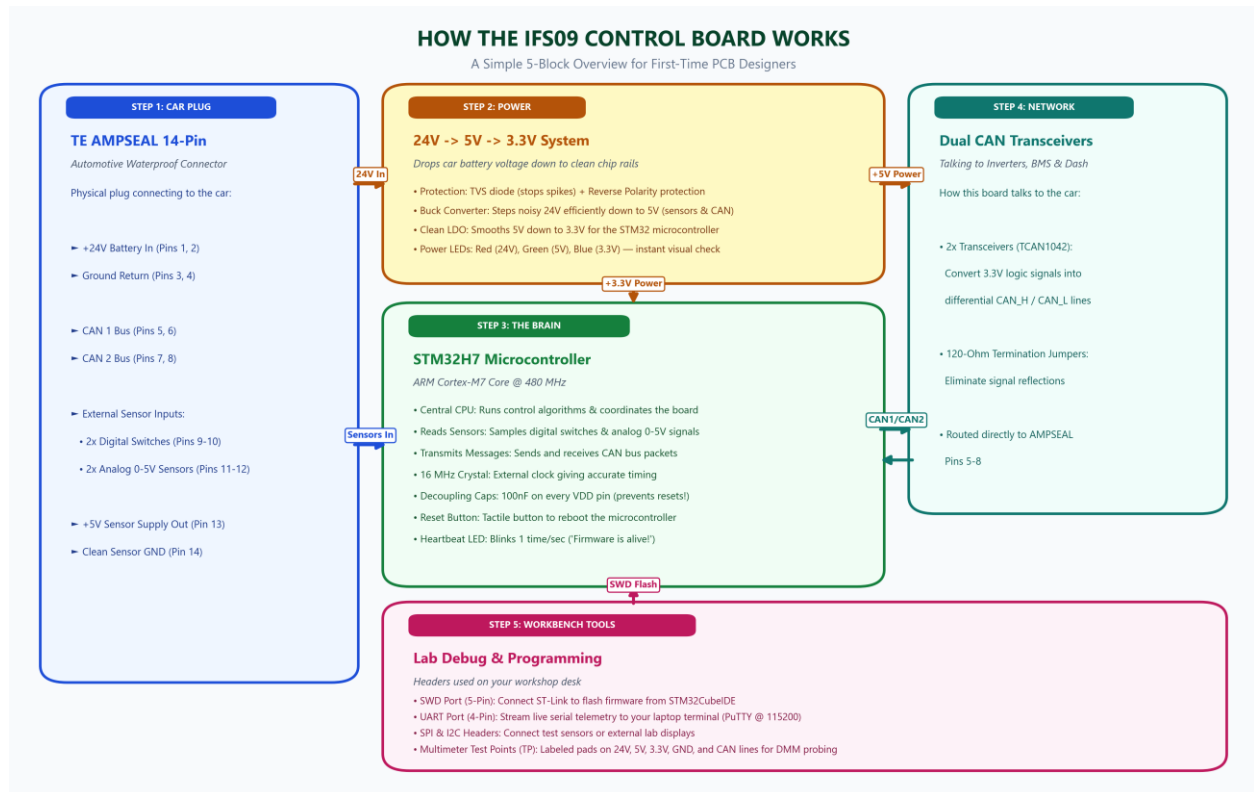


Figure 5.1: IFS09 Control Board Functional Block Diagram

## 5.1 Architecture Overview & Signal Flow

The board architecture is partitioned into five distinct, color-coded functional domains designed to ensure clean signal routing, eliminate ground loops, and prevent high-voltage noise from disturbing sensitive microcontroller operations:

- **Step 1: Car Plug (TE AMPSEAL 14-Pin - Royal Blue):** The sole physical interface connecting the PCB to the race car wiring harness. It receives the 24V low-voltage battery power, exposes dual CAN bus interfaces, brings in external analog and digital sensor signals, and provides a regulated 5V sensor power rail and clean sensor ground reference.
- **Step 2: Power System (24V -> 5V -> 3.3V - Warm Gold):** Converts the noisy 24V automotive battery into clean, stable DC voltage rails. It includes an input transient suppression stage (TVS diode and reverse-polarity protection), an efficient step-down Buck converter (24V to 5V) for high-current loads and external sensors, and a low-noise linear LDO regulator (5V to 3.3V) dedicated strictly to the STM32H7 core and sensitive logic.
- **Step 3: The Brain (STM32H7 Microcontroller - Fresh Green):** The high-performance computational core (ARM Cortex-M7 @ 480 MHz). It executes vehicle control firmware, samples analog inputs, manages dual CAN channels, and provides a visual 1 Hz heartbeat indicator to confirm normal system operation.
- **Step 4: Car Network (Dual CAN Transceivers - Teal Green):** Two dedicated automotive CAN transceivers (TCAN1042) that translate 3.3V logic signals from the microcontroller into rugged differential CAN\_H and CAN\_L communication buses, featuring 120-ohm split termination networks to eliminate signal reflections.
- **Step 5: Workbench Tools (Lab Debug & Programming - Rose Pink):** Workshop development headers including a 5-pin SWD programming port for ST-Link, a 4-pin UART header for live

serial telemetry streaming to a PC, SPI/I2C expansion headers, and dedicated surface test points for multimeter and oscilloscope diagnostics.

## 5.2 Functional Block Descriptions

- **Block 1: Car Plug (AMPSEAL 14):** All off-board connections route through TE Connectivity 776262-1: dual power pins (+24V, GND), dual CAN channels (CAN1, CAN2), 2x digital inputs, 2x 0-5V analog inputs, and a regulated +5V sensor supply with clean ground.
- **Block 2: Power Regulation Tree:** Features TVS surge clamping, reverse polarity P-MOSFET, switching buck (24V -> 5V) with >85% efficiency, low-noise LDO (5V -> 3.3V), and visual power LEDs (Red 24V, Green 5V, Blue 3.3V).
- **Block 3: The Brain (STM32H7 MCU):** STM32H7 processor with 16 MHz HSE crystal oscillator, 100 nF ceramic decoupling on every power pin, hardware reset button, and 1 Hz Heartbeat LED on PB0.
- **Block 4: Car Network (Dual CAN):** Dual TCAN1042 transceivers separating powertrain (Inverters/BMS) and telemetry/sensors with 120-ohm split termination and ESD diode arrays.
- **Block 5: Workbench Tools (Lab Debug):** Standard 2.54 mm pin headers for SWD firmware flashing (ST-Link), UART serial monitor (115200 baud), SPI/I2C expansion, and surface test points for multimeter verification.

## 6. Functional Subcircuits & Design Requirements

Break your schematic into clear functional subcircuits. For each block, pay close attention to what is VITAL (mandatory) and the EXTRA (automotive best practices).

### 6.1 Power Supply Subsystem (24V -> 5V -> 3.3V)

*Qué hace:* Receives the 24V DC supply from the car's Low Voltage battery, suppresses severe electrical noise and load-dump voltage spikes, and generates a regulated 5V rail for sensors and transceivers, plus a clean 3.3V rail for the microcontroller core.

- **VITAL:** High-efficiency step-down converter (input  $\geq 40V$ , output  $\geq 1.5A$ ) with power inductor and filter caps. (Tip: You can use a discrete synchronous buck like TPS54302 or the integrated OKI-78SR switching regulator module used in previous team designs!). 5V -> 3.3V clean LDO regulator (LM1117-3.3 / AP2112K). Bulk capacitors (10-47  $\mu F$ ) at the input/output of each regulator.
- **EXTRA:** High-power bidirectional TVS diode (SMCJ30A or SMBJ28A) across 24V input to swallow inductive spikes. P-MOSFET reverse polarity protection circuit. Resettable PPTC polyfuse on 24V line. LC pi-filter on MCU analog VDDA pin.

### 6.2 Microcontroller Unit (MCU — STM32H7)

*Qué hace:* The central computational brain of the board. It executes the control firmware, samples analog sensors, manages dual CAN buses, and commands external actuators.

- **VITAL:** STM32H7 series MCU (e.g. STM32H733 / H723). External 16 MHz or 25 MHz HSE crystal oscillator with matched load capacitors (12-18 pF). Reset push button with 100nF cap and 10k pull-up to 3.3V. Decoupling Capacitors (Rule #1 of Microcontrollers): Place a 100 nF ceramic

capacitor on EVERY SINGLE VDD and VDDA pin as close as physically possible with direct ground vias!

- **EXTRA:** BOOT0 pull-down resistor with jumper to 3.3V for DFU bootloader flashing. 1  $\mu$ F and 10  $\mu$ F ceramic reservoir capacitors near high-speed supply rails.

### 6.3 Dual CAN Bus Interfaces (CAN1 & CAN2)

*Qué hace:* Connects the board to the vehicle's high-speed differential CAN communication networks (Inverters, BMS, Dashboard, and Telemetry).

- **VITAL:** 2x automotive CAN/CAN-FD transceivers (TCAN1042 / MCP2562 / SN65HVD230) powered from 5V with 3.3V logic compatibility. 120-ohm bus termination on each channel selectable via 2-pin jumper. 100 nF decoupling capacitor across VCC/GND on each transceiver.
- **EXTRA:** Split termination network (two 60-ohm resistors in series with 4.7 nF cap to GND) for superior common-mode noise suppression. Automotive TVS diode arrays (PESD2CAN or NUP2105L) on CAN\_H and CAN\_L lines.

### 6.4 Automotive Connector: TE AMPSEAL 14-Pin Header

*Qué hace:* Rugged, waterproof physical interface connecting the PCB to the vehicle wiring harness.

| Pin #  | Signal Name             | What it connects to                                                            |
|--------|-------------------------|--------------------------------------------------------------------------------|
| 1, 2   | +24V_IN                 | 24V Power Input from LV Battery<br>(Two pins in parallel for current capacity) |
| 3, 4   | GND_PWR                 | Main Power Ground Return Plane                                                 |
| 5, 6   | CAN1_H / CAN1_L         | CAN Bus 1 (Vehicle Inverter & Primary Telemetry)                               |
| 7, 8   | CAN2_H / CAN2_L         | CAN Bus 2 (Sensors & Safety Subsystem)                                         |
| 9, 10  | DIG_IN1 / DIG_IN2       | Clamped Digital Inputs for sensors/switches                                    |
| 11, 12 | ANALOG_IN1 / ANALOG_IN2 | 0-5V Analog Sensor Inputs (with RC filter & clamping diodes)                   |
| 13     | 5V_OUT                  | Regulated 5V output to power external sensors                                  |
| 14     | GND_SIG                 | Clean analog signal ground reference                                           |

### 6.5 Expansion & Debug Headers (2.54 mm Pitch)

*Qué hace:* Allows bench programming, firmware debugging, and hardware expansion in the workshop.

- **VITAL:** SWD Programming Header (5-pin: 3V3, SWDIO, SWCLK, NRST, GND) for ST-Link flashing. Without this, your board is just an expensive decorative paperweight!

- **EXTRA:** SPI Header (6-pin), I2C Header (4-pin with 4.7k pull-up resistors), UART Header (4-pin) for serial debugging, and 8-pin digital GPIO header.

## 6.6 Diagnostic Status LEDs & Test Points

*Qué hace:* Allows anyone working on the car or in the workshop to instantly diagnose the board's health without needing an oscilloscope.

- **Diagnostic LEDs:** Power supply LEDs: Red on 24V ( $\geq 4.7k$  resistor), Green on 5V (1.5k), Blue on 3.3V (1k). MCU Heartbeat LED on PB0 (blinks once per second proving firmware is alive). CAN activity LEDs on transceiver TX/RX lines.
- **Test Points Everywhere:** Labeled test pads (min 1.0 mm pad diameter) for TP\_24V, TP\_5V, TP\_3V3, TP\_GND, TP\_CAN1\_H, TP\_CAN1\_L, TP\_Nrst, TP\_SWDIO, TP\_SWCLK.

## 7. Acceptance Testing Checklist (Post-Assembly Bench QA)

When your manufactured board arrives from the assembly factory, you will follow this step-by-step verification checklist:

- **Step 1 (Passive Short-Circuit Check):** Measure resistance across supply test points to GND with a multimeter. All readings must be  $> 10\text{ k}\Omega$  before power-up (no solder shorts).
- **Step 2 (Power-Up & Regulation):** Connect 24V bench power supply (current-limited to 200 mA). Verify  $V(\text{TP\_5V}) = 5.0V \pm 0.2V$ ,  $V(\text{TP\_3V3}) = 3.3V \pm 0.05V$ , and check that all power LEDs illuminate.
- **Step 3 (SWD Detection & MCU Flash):** Connect ST-Link to SWD header. Detect target in STM32CubeProgrammer and flash test firmware to blink the Heartbeat LED (PB0).
- **Step 4 (Dual CAN Loopback):** Connect CAN1 & CAN2 to a USB-CAN adapter (PCAN / CANable) and verify sending and receiving packets at 500 kbit/s with zero dropped frames.
- **Step 5 (Bus Probing):** Probe SPI, I2C, and UART with an oscilloscope to verify clean square wave signals.

## 8. What You Need to Submit: The Design Report

Once your schematic and PCB routing pass KiCad DRC and ERC with 0 errors, you will submit a formal Design Report containing:

- **1. Executive Summary:** Brief functional overview of your board and its role on the car.
- **2. Component Calculations & Justifications:** Buck converter component selection (inductor saturation current, diode rating, ripple calculation) and decoupling capacitor network.
- **3. Schematic Export:** Clean, high-resolution vector PDF schematic export (Schematics-1.0.pdf).
- **4. PCB Layout Showcase:** 2D layout captures showing the bottom ground plane and 3D renders showing mechanical clearance for the AMPSEAL connector.
- **5. Bill of Materials (BOM):** Complete spreadsheet with manufacturer part numbers (MPN), component packages, and supplier sourcing links.

## 9. Need Help? The Electronics Team is Here!

Remember: Nobody expects you to know everything on day one! Designing your first PCB is a collaborative learning experience, and asking questions is how every veteran on this team got started.

Whenever you get stuck on a circuit, have doubts about component footprints, or want someone to review your schematic before routing, reach out to your mentors:

- • Carlota Treviño
- • Félix González
- • Inés Pacheco
- • Andrés Sánchez de Ágreda (Electronics Technical Director)

You can find us in the ICAI workshop, on the team WhatsApp group, or by mentioning us during weekly meetings. Enjoy the journey from Mingo to Tronik!